Agent Foundations

ReAct Loop

The agent implemented in this exercise follows a minimal **ReAct-style loop: Reason → Act → Observe → repeat**. Instead of using an agent framework such as LangChain or LangGraph, the loop was implemented directly in Python using an OpenAI-compatible LLM API. The agent maintains a **messages** list containing the user request, assistant tool calls, and tool results. On every iteration, the model receives this conversation history and decides whether it needs to call a tool or can provide the final answer.

When the model requests a tool, the Python program parses the tool name and JSON arguments, executes the corresponding local Python function, and appends the result to the conversation as a tool message. The next iteration sends the updated history back to the model, allowing it to use the observation and decide what to do next. The loop continues until the model returns a normal text response or the **max_iterations** safeguard is reached. This was demonstrated with a multi-step weather task in which the agent called **get_weather** for Lahore and Karachi in separate iterations, then compared the returned temperatures and correctly concluded that Lahore was warmer by 7°C.

Tool Schemas Used

Two tools were defined using the OpenAI-compatible function-calling schema. The first tool, **calculator**, accepts an **expression** string and evaluates supported arithmetic operations including +, -, *, /, **, unary signs, and parentheses. The second tool, **get_weather**, accepts a **city** string and returns the temperature in Celsius and weather condition. Both tools use JSON Schema to specify their required inputs and set **additionalProperties** to false, making the expected tool arguments explicit.

The weather tool intentionally uses a small hardcoded database because it is a **stub**, not a real-time weather service. The calculator also performs defensive input validation so unsupported expressions or errors such as division by zero produce explicit error messages rather than silently returning incorrect results. Tool descriptions are important because the model does not see the underlying Python implementation. The tool name, description, and parameter schema therefore act as the interface contract that tells the model when a tool should be used and what arguments it expects.

Failure Modes Observed

Three failure/uncertainty cases were deliberately tested. First, requesting weather for **Atlantis** caused the weather tool to return an explicit error because no data was available. The agent then observed the error and produced an appropriate final response. Second, a request requiring an unavailable capability, such as currency conversion, was handled without inventing a nonexistent tool; the model explained that no suitable tool was available. Third, the ambiguous request **“Is it nice outside?”** caused the model to ask for a city or location instead of guessing.

Additional potential failure modes include infinite loops, malformed tool arguments, tool exceptions, and excessive conversation-history growth. The implementation mitigates these through a **max_iterations** limit, defensive JSON parsing, **try/except** handling around tool execution, explicit error messages, and a separate **working_memory** structure for tracking iterations, tool calls, observations, and errors.

Overall, this exercise demonstrated that an agent is fundamentally a model-driven loop around **tool selection, execution, observation, state, and stopping conditions**. Frameworks such as LangChain, LangGraph, and CrewAI provide abstractions for these same mechanisms, making larger agent systems easier to build and maintain without hiding the underlying concepts.